PrivQ: A Python Framework for Modeling Privacy-Preserving Query Processing

Alex Garza
Department of Computer Science
Tufts University
Medford, Massachusetts, USA
Alex.Garza@tufts.edu

Johes Bater
Department of Computer Science
Tufts University
Medford, Massachusetts, USA
Johes.Bater@tufts.edu

Abstract Secure multi-party computation systems for relational analytics suffer from severe performance overhead caused by oblivious execution and intermediate result padding. Recent systems such as Shrinkwrap [1] and ORQ [2] propose techniques to mitigate these costs through differential privacy and optimized join-aggregation execution strategies. Reasoning about the relative benefits of these techniques requires either a full MPC deployment or analytic reasoning over the underlying cost model.

This work presents PrivQ, a Python framework for pedagogically exploring the privacy-performance tradeoffs introduced by Shrinkwrap and ORQ. PrivQ contributes (i) a SQL-text rewriting layer that exposes the published-system mechanisms (column-order normalization, structural padding, truncated-Laplace DP resize, DP-noisy join reordering) as a small Python API, and (ii) a plan-layer simulator that propagates true and padded cardinalities through a query plan under three policies (fully oblivious, Shrinkwrap-style DP resize, and an ORQ-style join-aggregation bound) and reports per-operator simulated cost under the Shrinkwrap unit cost model.

We do not claim a new MPC system or a novel privacy mechanism. The framework's value is making the cost equations of Shrinkwrap and ORQ executable: cascading $n \to n^2 \to n^3$ blowup is reproduced on a synthetic healthcare plan, and a sweep over ϵ reveals a crossover at which the ORQ-style sum bound overtakes Shrinkwrap-style resize, a regime change visible in the simulator but not immediately obvious from the closed-form cost model. PrivQ is published on PyPI as a teaching artifact for secure-database courses and as a pre-deployment exploration tool for analysts considering DP- or ORQ-style backends.

I. Introduction

The storage and analysis of sensitive data has grown dramatically in recent years. Organizations increasingly wish to collaborate on analytics while preserving privacy guarantees for the underlying data.

A concrete example makes the stakes clear. A broker-dealer bank publishes a daily list of the assets it is willing to trade, aggregated across all of its clients, so that buyers and sellers can be matched efficiently [3]. The list is meant to reveal nothing about any single client. Yet when one client holds most of the position in a given asset, that client's buying and selling drives the published aggregate, and a competitor watching the list day after day can reconstruct the client's strategy. The aggregate that was supposed to protect each participant instead leaks the activity of the one that matters most. The same shape of problem recurs whenever multiple parties pool data for a shared computation: a hospital consortium counting patients across institutions, or a set of banks jointly screening for fraud. Each party wants the answer to the joint query without exposing its own records, and a naively computed aggregate can still betray an individual contributor.

Secure multi-party computation (MPC) provides a mechanism for jointly evaluating computations over distributed datasets without revealing the private inputs of participating parties.

Although MPC provides strong privacy guarantees, practical deployment for relational analytics remains difficult due to the overhead of oblivious execution. Secure query engines must hide memory access patterns, execution traces, and intermediate result sizes, and they accomplish this by padding every intermediate operator output to its worst-case size. Shrinkwrap [1] demonstrates this with a cascading join example in which intermediate cardinalities grow from n to n^2 and eventually to n^3, dramatically increasing execution cost.

Two recent systems attack this overhead from complementary angles. Shrinkwrap [1] introduces differential-privacy-based resizing to release noisy intermediate cardinalities, replacing worst-case padding with provably (ϵ, δ)-differentially-private estimates. ORQ [2] introduces an oblivious join-aggregation operator that fuses sort, join, and aggregation in a single oblivious pass, avoiding the materialization of large intermediate joins entirely. Both systems are difficult to experimentally study because each requires a full MPC stack, a distributed deployment, and cryptographic runtime infrastructure.

This paper presents PrivQ, a Python framework that makes the cost equations of Shrinkwrap and ORQ executable. PrivQ is not a cryptographic MPC engine and does not claim a new privacy mechanism; it is a teaching and exploration framework that lets a user compose the published-system mechanisms on a query plan and observe per-operator cardinality and cost behavior under each one. The point of writing simulator code rather than evaluating cost expressions directly is twofold: (a) the simulator exposes the same Python objects (QueryPlan, Filter, Join, three Policy classes) that a downstream MPC-backed planner would manipulate, so plan rewrites and policy choices made in PrivQ can be lifted out unchanged; and (b) the policies have stochastic components (truncated-Laplace resize, DP-noisy join reordering) whose distribution of outcomes is more naturally inspected by sampling than by closed-form analysis. The evaluation in §VII uses this property to expose a privacy-budget regime where the policy ordering flips, a fact the cost model implies but does not make immediately visible.

PrivQ provides two major components:

1) A SQL rewriting layer that performs structural query transformations inspired by Shrinkwrap and ORQ.
2) A plan-layer simulator that models intermediate cardinality growth and simulated execution cost under different secure query execution policies.

The contributions of this work are pedagogical and exploratory rather than cryptographic or algorithmic:

- A small Python API that surfaces the four published-system mechanisms (Shrinkwrap-style DP resize, ORQ-style join-agg bound, column-order normalization, DP-noisy join reordering) behind a uniform interface, so that a student or analyst can compose them on a SQL query in a few lines.
- A plan-layer simulator that propagates true and padded cardinalities through a relational plan under three policies and reports per-operator simulated cost under the Shrinkwrap unit cost model (Table 2 of [1]).
- An evaluation that reproduces the cascading-blowup behavior described by Shrinkwrap on a synthetic healthcare plan, and identifies a privacy-budget regime where the policy ordering between DP resize and the ORQ-style sum bound flips (§VII).
- A PyPI-published, MIT-licensed implementation tagged v1.3.0-paper, intended for classroom use and for pre-deployment exploration of secure-execution strategies.

We are explicit that PrivQ is not a cryptographic engine, does not execute MPC, and does not produce wall-clock measurements; the simulator reports trends in a unit-cost model, not predicted runtime (§V-E, §IV).

II. Background

A. Secure Multi-Party Computation

Secure multi-party computation allows multiple parties to jointly compute a function over their private inputs without revealing those inputs to each other. In secure relational query processing systems, relational operators such as filters, joins, and aggregations are evaluated over encrypted or secret-shared data.

To preserve privacy, secure query execution must be oblivious. Oblivious execution ensures that memory access patterns, control flow, and intermediate result sizes do not reveal information about the underlying data.

Unfortunately, oblivious execution introduces significant overhead. In a traditional database engine, operator selectivity naturally reduces intermediate result sizes. In oblivious systems, however, intermediate results must often be padded to their worst-case size.

For example, consider the following query:

```
1 SELECT DISTINCT pid
2 FROM demographic de,
3      diagnosis di,
4      medication m
5 WHERE di.diag = 'hd'
6   AND m.med = 'aspirin'
7   AND di.code = m.code
8   AND de.pid = m.pid;
```

Listing 1. Cascading three-way join example.

Under oblivious execution, filters may preserve an intermediate size of n, while joins over two size-n relations may produce padded outputs of size n^2. Subsequent joins may then grow to n^3. This cascading blowup dramatically increases execution cost.

B. Differential Privacy

Differential privacy provides a mechanism for releasing aggregate information about a dataset while limiting information leakage about individual records. The calibrated tradeoff it offers, accepting a bounded, quantifiable amount of leakage in exchange for utility, is not only of theoretical interest: it underpins deployed systems, including the Atlas-X axe-obfuscation system [3] now running in production at a major financial institution.

A randomized mechanism f satisfies (ϵ, δ)-differential privacy if:

$$\Pr[f(D) \in O] \leq e^{\epsilon} \Pr[f(D') \in O] + \delta \tag{1}$$

for neighboring datasets D and D' differing in a single record. Shrinkwrap leverages differential privacy to release noisy intermediate cardinalities during secure query execution. Rather than padding every intermediate result to its worst-case size, Shrinkwrap publishes a noisy estimate of the true cardinality and resizes the secure array accordingly.

The specific mechanism, defined in Bater et al. [1] Def. 4, is a truncated Laplace mechanism. Given true cardinality c and sensitivity Δc, the mechanism samples noise η from a Laplace distribution with scale $\Delta c/\epsilon$ shifted by an offset η_0 such that $\Pr[\eta < \Delta c] \leq \delta$, and releases

$$\tilde{c} \;=\; c + \max(\eta, 0). \tag{2}$$

The shift guarantees that the released size almost certainly exceeds the true cardinality, so the secure-array truncation that follows the resize does not drop real rows. Throughout this paper we refer to this mechanism as DP-resize.

C. Shrinkwrap

Shrinkwrap [1] introduces differential privacy into secure relational query processing by replacing exhaustive worst-case padding with noisy intermediate cardinalities.

Rather than padding every operator output to its maximum possible size, Shrinkwrap computes a differentially private estimate of the intermediate cardinality and resizes the secure array accordingly.

This significantly reduces the computational overhead associated with secure query execution while maintaining differential privacy guarantees.

The core idea of Shrinkwrap is to trade limited, controlled information leakage for dramatic performance improvements.

D. ORQ

ORQ [2] focuses on reducing the cost of oblivious joins and aggregations in MPC systems.

A major observation in ORQ is that many analytical workloads do not require materializing the full Cartesian product of intermediate joins. Instead, joins and aggregations can often be fused together.

ORQ introduces a composite join-aggregation operator that performs aggregation during the join process itself. This keeps intermediate results bounded and avoids many of the cascading costs associated with oblivious execution.

ORQ also introduces optimized oblivious sorting and aggregation protocols that achieve improved asymptotic performance for many practical workloads.

III. The MPC Landscape

Shrinkwrap and ORQ are recent points in a longer trajectory of secure-computation research. Because PrivQ models the cost and leakage behavior of that trajectory rather than reimplementing it, we situate the two systems within the broader state of MPC: the cryptographic primitives that make oblivious computation possible, the line of work that lifted those primitives to relational analytics, and the recent move of MPC and differential privacy from the laboratory into production. Figure 1 sketches the secret-sharing setting these systems share.

A. Cryptographic Foundations and Adversary Models

Secure multi-party computation rests on two classical paradigms. Garbled circuits, originating with Yao, let two parties evaluate a Boolean circuit on encrypted inputs; secret sharing, as in the GMW, BGW, and SPDZ lines, splits each value into shares held by different parties so that no proper subset learns anything, and evaluates arithmetic or Boolean gates by local computation interleaved with communication. Modern systems mix these paradigms and are characterized along two axes that any reader of an MPC paper must keep straight. The first is the adversary's behavior: a semi-honest (honest-but-curious) adversary follows the protocol but inspects whatever it sees, whereas a malicious (active) adversary may deviate arbitrarily. The second is the corruption threshold: an honest-majority protocol assumes fewer than half the parties are corrupt, while a dishonest-majority protocol tolerates all-but-one. These choices set both the achievable security and the cost. Dalskov et al. [6] illustrate the frontier of the honest-majority regime, giving a fully secure protocol with guaranteed output delivery against an active adversary corrupting fewer than one third of parties over any ring, with constant amortized communication per multiplication gate. The taxonomy matters for PrivQ because the cost and leakage a secure query engine exhibits, the very quantities PrivQ models, are a function of where on these axes the engine sits; PrivQ follows the semi-honest assumption of Shrinkwrap [1] and ORQ [2] (§IV).

B. From Primitives to Relational Analytics

Raw MPC primitives evaluate circuits, not SQL. Bridging that gap is the problem that secure relational query processing solves, and it is the immediate lineage of PrivQ. SMCQL [4] introduced the private data network: a federation of mutually distrustful data owners served by an honest broker that compiles SQL into secure-computation primitives and executes it obliviously, evaluated on real clinical data. Because circuit-level MPC is expensive, SMCQL's optimizer minimizes the work done under MPC by splitting a query into cleartext-executable and secure slices. Conclave [5] pushed this idea to larger inputs, using coarse-grained trust annotations to run most of a relational workload in cleartext and reserve MPC for the operators that genuinely cross trust boundaries, generating code for Spark and for the Sharemind and Obliv-C backends. Both systems confront the obstacle that motivates this paper: an oblivious operator cannot reveal how many rows survive a filter or a join, so it pads its output to the worst-case size, and these padded sizes cascade. Shrinkwrap [1] attacks the padding directly with differentially private resizing, and ORQ [2] attacks it by fusing join and aggregation so the large intermediate is never materialized. PrivQ's plan-layer simulator is a model of exactly this padding behavior across the SMCQL-to-ORQ design space.

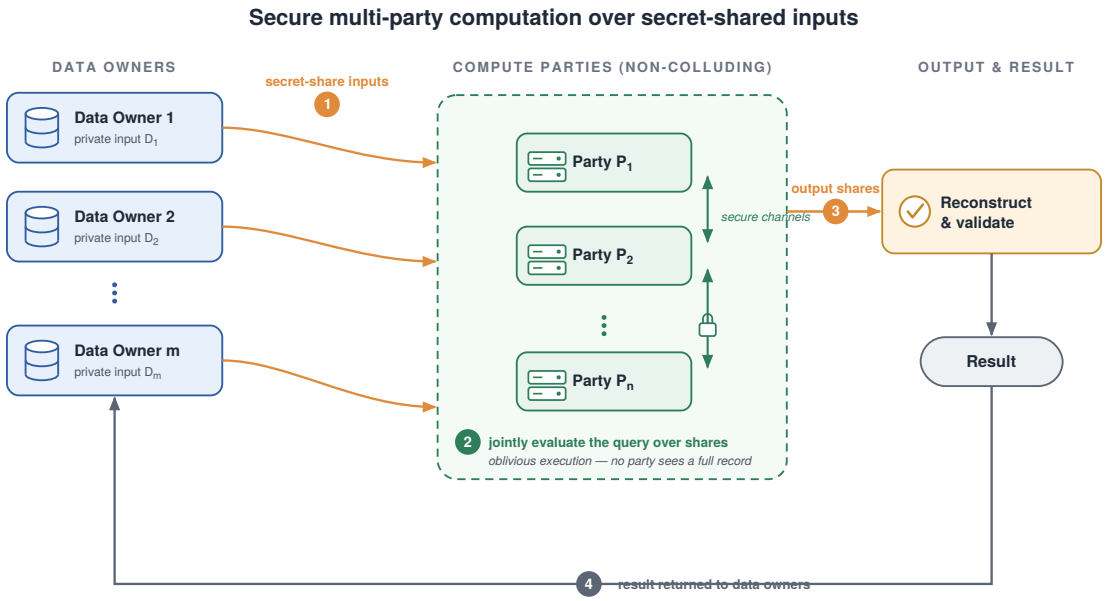


Fig. 1. Secure multi-party computation over secret-shared inputs. Data owners secret-share their private inputs to a set of non-colluding compute parties (1); the parties jointly evaluate the query over shares, executing obliviously so that no party observes a full record (2); the output shares are reconstructed and validated (3); and the result is returned to the data owners (4). PrivQ models the cost and cardinality leakage of step (2) without implementing the underlying cryptography.

C. Composing MPC with Differential Privacy

A parallel thread composes MPC with differential privacy so that the aggregate a protocol releases is itself privatized, rather than only the inputs being hidden during the computation. Much of this work targets federated learning, where parties jointly train a model without sharing data. Byrd and Polychroniadou [8] combine secure aggregation with differential privacy so that a coordinating server learns only a noisy aggregate gradient, never any client's update, demonstrated on credit-card fraud data; SMPAI [9] generates the differentially private noise itself in a distributed, encrypted manner so that no single party, including the server, sees an individual noisy weight; and Das et al. [10] reduce the communication cost of such DP-MPC training by orders of magnitude through fixed-point arithmetic and locally sampled discrete-Gaussian noise. Secure aggregation, the primitive underneath much of this work, has itself been streamlined for the single-server, client-dropout setting [11]. PrivQ sits at the intersection these systems define, modeling DP applied to oblivious computation, but in the relational-query setting of Shrinkwrap rather than the gradient-aggregation setting of federated learning.

D. The Deployment Frontier

For most of its history MPC was confined to research prototypes and benchmarks, but that has begun to change, which is part of why a teaching artifact for these techniques is now worthwhile. Prime Match [7] is a two-party secure-computation system, with malicious security and no preprocessing, that matches buy and sell orders between a bank's clients without revealing unmatched interest; it is reported as the first MPC system deployed in production at a major bank. Atlas-X [3], discussed further in §X, similarly runs a differentially private mechanism live across three financial regions. PrivQ does not compete with these systems; it gives students and analysts a lightweight way to reason about the cost and leakage tradeoffs that systems of this kind now face in practice.

IV. Threat Model and Security Assumptions

We state the threat model before describing PrivQ's design so that the reader can judge each mechanism against the leakage it is meant to address. PrivQ is a simulation framework, not a cryptographic system; the purpose of this section is to make precise what PrivQ does and does not model.

a) Adversary: We follow Bater et al. [1] and Baum et al. [2] in considering a semi-honest (honest-but-curious) adversary that controls a subset of computing parties or observes the secure-computation transcript, consistent with the adversary taxonomy of §III. The adversary follows the protocol but may inspect intermediate state, including the sizes of secure arrays produced by each operator.

b) Leakage channel modeled: The leakage channel that PrivQ models is the cardinality of each operator's output. Under fully oblivious execution, this leakage is eliminated by padding every output to its worst-case size;

the cost is the cascading blowup described in §I. A DP-resize policy (§V-E) trades a calibrated, (ϵ, δ)-bounded amount of cardinality leakage for a dramatic reduction in padding cost. An ORQ-style join-aggregation policy reduces padding to $|A| + |B|$ without releasing any size information at all, by collapsing the join and aggregation into one oblivious pass.

c) Why aggregate leakage matters: database reconstruction: The reason a calibrated bound on cardinality leakage is worth paying for is that aggregate releases, even ones that name no individual, can be inverted to recover the underlying records. A database reconstruction attack treats a set of published statistics as a system of equations and solves for a microdata table consistent with them; Garfinkel et al. [13] give an accessible account of how the U.S. Census Bureau found its published tabulations vulnerable to exactly this attack, motivating the Bureau's adoption of differential privacy for the 2020 decennial census. Reconstruction is distinct from reidentification, which links an anonymized record to a known individual using auxiliary data, as in the de-anonymization of the Netflix Prize dataset [14]; conflating the two can overstate risk, and the strength of the reconstruction threat is itself debated, with Muralidhar and Domingo-Ferrer [15] arguing that conventional disclosure-control techniques can avert reconstruction without resorting to differential privacy. PrivQ does not perform reconstruction or model a specific attack; the relevance here is that the intermediate cardinalities PrivQ tracks are precisely the kind of aggregate signal whose uncontrolled release this literature shows can leak individual information, which is why bounding that leakage, rather than ignoring it, is the design goal of DP-resize.

d) Leakage channels not modeled: PrivQ does not model lower-level cryptographic leakage channels: it does not analyze the message-pattern leakage of a specific MPC protocol, the side-channel leakage of an oblivious-array implementation, or the timing leakage of network communication. PrivQ also does not implement secret sharing, garbled circuits, or any other cryptographic primitive; it reasons only about the cardinalities that a hypothetical secure-execution layer would emit at each operator.

e) Privacy budget composition: A meaningful gap between PrivQ and a deployed DP system is that the DP-resize policy (§V-E) consumes its (ϵ, δ) budget independently at every operator. The simulator does not enforce a global privacy budget across the chain via sequential composition (Shrinkwrap Theorem 1) or via the optimal budget-allocation strategy of Bater et al. [1] §5. Reported per-operator cardinalities are correctly (ϵ, δ)-differentially private as releases in isolation, but the total privacy cost of a full plan with ℓ operators is $(\ell\epsilon, \ell\delta)$ under naive composition. Closing this gap is the most important next step for the simulator (§XII).

f) Intended use: PrivQ is intended for research experimentation, classroom demonstrations, and the pre-deployment exploration of secure-query-execution strategies. It is not appropriate as a deployment substitute for a real MPC system.

V. System Design

A. Overview

PrivQ is designed as a lightweight experimentation framework for studying privacy-preserving query execution strategies. Figure 2 gives an overview of the system: a SQL rewriting layer transforms input SQL with column normalization, structural padding, DP-noisy LIMIT insertion, and DP-noisy join reordering; a plan-layer simulator evaluates relational QueryPlans under three padding policies and returns a per-operator SimulationReport.

The two layers are intentionally independent and consume different inputs. The SQL rewriting layer takes a SQL string and returns a SQL string; the plan-layer simulator takes a QueryPlan object (built programmatically, not parsed) plus a list of Table cardinalities and returns a structured report. They share C++ primitives: the truncated-Laplace mechanism used by dp_resize() is the same one used by DPResizePolicy, and the noisy table-size estimator used by privq_join() is the same one used by OrqPolicy.reorder_tables(). However, no path inside PrivQ runs SQL text through the simulator or emits SQL text from a simulator report. The end-to-end example in §VIII shows the intended composition pattern: the user runs the simulator to choose a policy and a noisy LIMIT, then calls dp_resize() to imprint that LIMIT on a SQL string bound for a federated backend.

The system consists of two major layers:

1) A SQL-text rewriting layer.
2) A plan-layer simulation engine.

The SQL rewriting layer performs transformations inspired by Shrinkwrap and ORQ, including:

- Column-order normalization.
- Structural query padding.
- Differentially private LIMIT insertion.
- Differentially private join reordering.

The plan-layer simulator models relational query plans under multiple secure execution policies and estimates intermediate cardinalities and simulated execution cost.

B. SQL Rewriting Layer

The SQL rewriting layer provides lightweight transformations for experimenting with query normalization and privacy-aware query rewriting.

1) Column Normalization: The orq() transformation sorts SELECT columns alphabetically to normalize column-order leakage.

```
1 import PrivQ as p
2
3 sql = "SELECT salary, name, age FROM employees"
4 print(p.orq(sql))
```

Listing 2. Calling the orq() column-normalization transformation.

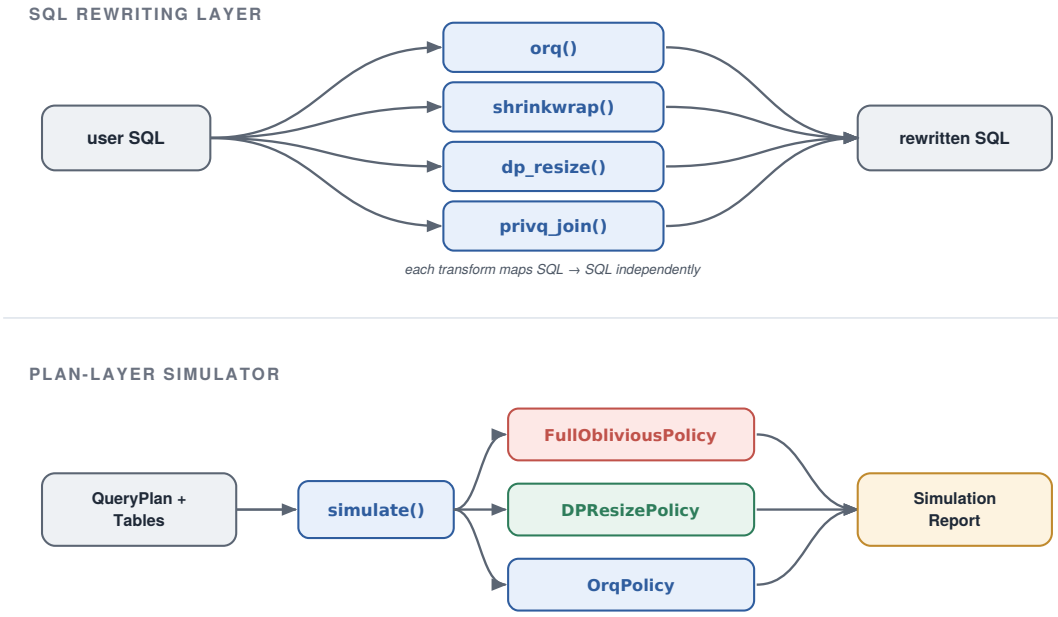


Fig. 2. PrivQ system architecture. The SQL rewriting layer (top) provides four text-level transformations that can be applied to a SQL query. The plan-layer simulator (bottom) evaluates a relational QueryPlan under three padding policies and returns a structured SimulationReport.

```
1 SELECT age, name, salary FROM employees
```

Listing 3. Output of the above program.

2) Structural Padding: The shrinkwrap() transformation rewrites a SQL string by appending dummy columns, additional subqueries, padding comments, and dummy joins. We caution upfront that this transformation is not an implementation of the Shrinkwrap mechanism of Bater et al. [1]: Bater's Shrinkwrap is about intermediate-cardinality leakage during secure execution, while this transformation operates purely on SQL text. We preserve the function name for continuity with the public PyPI release and the project's documentation, but the threat the text-layer pads address is a text-channel observer who inspects raw SQL bound for a federated backend, not an MPC adversary with operator-level transcript visibility. The Shrinkwrap mechanism proper is modeled by DPResizePolicy in the plan-layer simulator (§V-E). The text-layer transformation is included as a teaching aid and as a hook for future text-channel hardening work; we revisit this limitation in §IX.

3) Differentially Private Resize: The dp_resize() transformation appends a noisy LIMIT clause using a truncated Laplace mechanism.

```
1 p.dp_resize(sql,
2             true_count=100,
3             epsilon=1.0,
4             delta=1e-6)
```

Listing 4. Calling dp_resize() to append a DP-noisy LIMIT.

```
1 SELECT ... LIMIT 112
```

Listing 5. Possible output (exact value is random).

The C++ implementation of dp_resize() directly evaluates Eq. (2) and rewrites the SQL in place, replacing or appending the trailing LIMIT clause while preserving any trailing semicolon.

4) Differentially Private Join Reordering: The privq_join() transformation reorders a list of input tables from smallest to largest based on noisy size estimates drawn from a catalog. True row counts are never released; each catalog entry is perturbed with $\mathrm{Lap}(1/\epsilon)$ noise before being inserted into the internal min-heap. The relevant sensitivity here is $\Delta = 1$: adding or removing a single record changes any one table's row count by at most one, so the Laplace scale $1/\epsilon$ is calibrated to ϵ-DP on a single-record-neighboring catalog. The privacy semantics of privq_join() are therefore distinct from the per-operator resize semantics of DPResizePolicy: the former releases noisy statistics about the inputs read from an offline catalog, while the latter releases noisy statistics about intermediate cardinalities computed during query

execution.

```
p.privq_join(
    ["orders", "users", "products"],
    catalog=[("users",    100),
             ("orders",   200),
             ("products", 50)],
    epsilon=0.5,
)
```

Listing 6. DP-noisy join reordering from a catalog.

The library returns the noisy ordering as data; an upstream layer with a real SQL parser is expected to apply it to the FROM/JOIN clause. A persistent estimator object (p.estimator()) is also provided so a catalog can be loaded once and reused across many queries.

C. Plan-Layer Simulator

The PrivQ simulator models relational query execution using chains of relational operators.

Supported operators include:

- Filter
- Join
- Aggregate

Each query plan is evaluated under multiple secure execution policies.

```
from PrivQ.plan import (
    Table,
    QueryPlan,
    Filter,
    Join,
    Aggregate,
    FullObliviousPolicy,
    DPResizePolicy,
    OrqPolicy,
)
```

Listing 7. Importing the plan-layer simulator.

D. Execution Policies

PrivQ currently supports three execution policies.

TABLE I
PrivQ execution policies

Policy	Filter	Join	Aggregate
FullOblivious	input	$n \cdot m$	input
DPResize	noisy	noisy	noisy
OrqPolicy	input	$n + m$	input

1) FullObliviousPolicy: The fully oblivious policy pads intermediate results to worst-case size.

For joins, this produces Cartesian-style padding of size $n \cdot m$.

2) DPResizePolicy: The differential privacy policy models Shrinkwrap-style noisy resizing.

Intermediate cardinalities are estimated using a truncated Laplace mechanism.

3) OrqPolicy: The ORQ-inspired policy models join-aware aggregation strategies. Rather than padding joins to the Cartesian product $n \cdot m$, joins are bounded to approximately $n + m$, reflecting ORQ's observation [2] that a join followed by an aggregation can be computed as a single oblivious sort-then-aggregate pass over the concatenated inputs. OrqPolicy.reorder_tables() additionally calls privq_join() to apply a DP-noisy smallest-first ordering of the input tables.

E. Cost Model

The simulator does not perform real MPC execution. Instead, it propagates an effective intermediate cardinality through the operator chain and converts those cardinalities into a simulated I/O cost using the unit cost model of Bater et al. [1] (Table 2). Let $|A|$ and $|B|$ denote the cardinalities of an operator's left and right inputs and $c_r = c_w = 1$ denote unit read and write costs. The per-operator costs are then

$$C_{\text{filter}}(|A|) = |A|\, c_r + |A|\, c_w, \tag{3}$$

$$C_{\text{join}}(|A|, |B|) = |A|\, c_r + |A||B|\, c_r + |A||B|\, c_w, \tag{4}$$

$$C_{\text{agg}}(|A|) = |A|\, c_r + c_w, \tag{5}$$

$$C_{\text{sort}}(|A|) = |A| \log_2 |A|\, (c_r + c_w). \tag{6}$$

The total simulated plan cost is the sum of per-operator costs.

The intermediate cardinalities themselves are produced by the active policy. Under fully oblivious execution, every operator pads to its worst-case output size; for a join this is the Cartesian product

$$|O|_{\text{pad}} = |A| \cdot |B|, \tag{7}$$

which is the source of the $n \to n^2 \to n^3$ cascading blowup. The DP-resize policy instead releases a noisy estimate

$$|O|_{\text{dp}} = |O|_{\text{true}} + \text{Lap}(\Delta/\epsilon), \tag{8}$$

which we implement using the truncated variant in Eq. (2) so that $|O|_{\text{dp}} \geq |O|_{\text{true}}$ with probability $1 - \delta$. The ORQ policy bounds join outputs to $|A| + |B|$, the join-aggregation upper bound that arises from ORQ's sort-then-fuse construction (Baum et al. [2], §3.3 introduces the basic Join-Agg operator producing a table of size $|L| + |R|$, and §3.5–3.6 extend it to grouped and finalized aggregations).

Two consequences of this cost model are worth stating explicitly. First, because joins dominate cost as $O(|A||B|)$, any policy that reduces the join output size by even a small constant factor produces dramatic reductions in total cost. Second, the simulator reports trends and relative growth rather than wall-clock MPC runtime; absolute numbers should be read as ratios across policies on the same plan, not as predictions of MPC execution time.

VI. Implementation

PrivQ is implemented in Python (compatible with Python $\geq$ 3.9) with three C++17 extension modules built using pybind11. The Python source under src/PrivQ/, including the plan-layer simulator and the three policies, totals 1,043 lines; the C++ extensions under src/PrivQ/{ORQ,ShrinkWrap,PrivQJoin}/ add 1,216 lines including the truncated Laplace mechanism, the ten structural pads used by the text-layer shrinkwrap(), and the noisy table-size estimator used by privq_join(). Counts are reported with wc -l on the v1.3.0-paper tag.

The project is published on PyPI as PrivQ; the experiments in Section VII were produced using version 1.3.0, tagged in the public git repository as v1.3.0-paper. The full source, build instructions, and a runnable Jupyter walkthrough of the API are available at https://github.com/alex11garza/PrivQ.

The project structure is shown in Listing 8.

```
PrivQ/
 src/PrivQ/
      __init__.py
      ORQ/
      ShrinkWrap/
      PrivQJoin/
      plan/
 examples/
 pyproject.toml
 README.md
```

Listing 8. PrivQ project structure.

The three C++ extension modules (_orq, _shrinkwrap, _privq_join) implement the column-sort logic, all ten ShrinkWrap structural pads plus the truncated Laplace mechanism backing dp_resize(), and the noisy table-size estimator backing privq_join() respectively. The plan-layer simulator is pure Python and depends only on the standard library.

VII. Evaluation

A. Experimental Setup

We evaluate PrivQ using synthetic relational workloads inspired by the examples presented in Shrinkwrap and ORQ.

Experiments focus on:

- Intermediate cardinality growth.
- Simulated execution cost.
- Differential privacy parameter tradeoffs.

B. Cascading Blowup

We first evaluate the impact of fully oblivious execution on intermediate cardinalities. The query plan in Listing 9 models a three-way healthcare join over three input tables: diagnosis (10,000 rows), medication (20,000 rows), and demographic (5,000 rows). The filter selectivity is 0.01 (one in a hundred diagnoses is for heart disease), and the joins use K-multiplicities of 3 and 1 respectively. This is deliberately the same three-table schema (diagnosis, medication, demographic) used as the canonical Aspirin Count motivating example in Shrinkwrap [1], restated as a QueryPlan; we reproduce it here so that PrivQ's reported numbers can be cross-checked against the cardinality behavior described in Bater §3. Concretely, the filter selects 100 surviving diagnoses; the first join with medication at multiplicity 3 expands those to 300 (each diagnosis matches three medication rows on average); and the second join with demographic at multiplicity 1 keeps 300 (every patient in the cohort has exactly one demographic record, a pure semi-join).

```
plan = QueryPlan([
    Filter("diagnosis",
           predicate="diag == 'hd'",
           selectivity=0.01),
    Join("", "medication",
         on="code",
         multiplicity=3),
    Join("", "demographic",
         on="pid",
         multiplicity=1),
    Aggregate(group_by="pid",
              op="distinct"),
])
```

Listing 9. Three-way healthcare query expressed as a QueryPlan.

Under fully oblivious execution, padded intermediate cardinalities grow catastrophically.

TABLE II
Intermediate growth under fully oblivious execution

Operator	True	Padded
Filter	100	10,000
Join 1	300	200,000,000
Join 2	300	1,000,000,000,000

This demonstrates the cascading $n \to n^2 \to n^3$ behavior described in Shrinkwrap. Figure 3 plots the same numbers on a logarithmic axis; the gap between true and padded cardinalities spans more than ten orders of magnitude by the second join.

C. Differential Privacy Resize

We next evaluate the DPResize policy. The headline table is reported in two forms: as the per-operator privacy budget that the simulator actually consumes, and as the global budget that the full four-operator plan in Listing 9 accumulates under naive sequential composition (Shrinkwrap Theorem 1; see also §IV).

The two policies make different privacy claims and the comparison is therefore not a strict speedup contest. DPResize achieves a roughly seven-order-of-magnitude reduction over the fully-oblivious baseline by releasing differentially private intermediate cardinalities, paying per-operator (ϵ, δ) in observable size at each operator; under naive sequential composition the four-operator plan

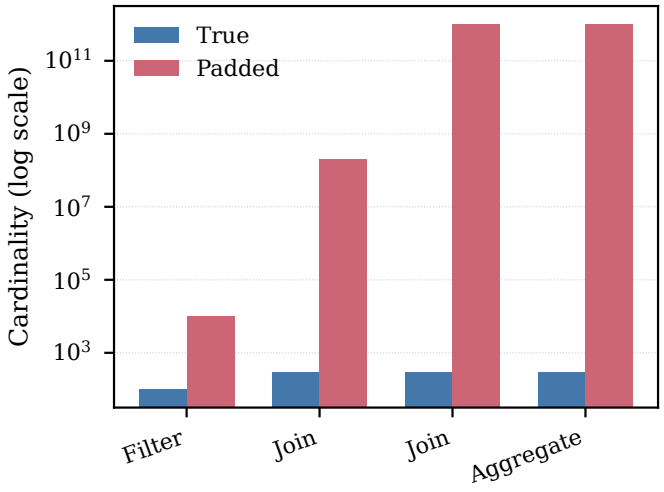


Fig. 3. True versus padded cardinality at each operator under fully-oblivious execution, on the plan in Listing 9.

TABLE III
Simulated execution cost and leakage by policy on the plan in Listing 9. DPResize is reported at a per-operator budget of $(\epsilon, \delta) = (1, 10^{-6})$; the four-operator plan accumulates global $(4, 4 \times 10^{-6})$ under sequential composition.

Policy	Total Cost	Speedup	Information leaked
FullOblivious	8.27×10^{13}	$1\times$	none
DPResize / op	7.96×10^{6}	$\approx 1.04 \times 10^{7}\times$	per-op (ϵ, δ)
DPResize plan	7.96×10^{6}	$\approx 1.04 \times 10^{7}\times$	$(4\epsilon, 4\delta)$ global
OrqPolicy	7.01×10^{8}	$\approx 1.18 \times 10^{5}\times$	none†

†OrqPolicy releases no DP samples; the secure array sizes are deterministically $|A| + |B|$ at each join. If the input table sizes themselves are sensitive, the optional DP-noisy join reordering of privq_join() adds ϵ-DP leakage on the catalog, distinct from any intermediate-result leakage.

in Listing 9 therefore costs a global $(4\epsilon, 4\delta) = (4, 4 \times 10^{-6})$. ORQ's join-aggregation bound buys a five-order-of-magnitude reduction by replacing $|A| \cdot |B|$ with the deterministic bound $|A| + |B|$ at every join, releasing no DP sample at all. A fair side-by-side reading of the table is therefore: DPResize pays a small, calibrated privacy cost for two extra orders of magnitude over an ORQ baseline that pays no privacy cost. The two strategies attack different terms in the cost model and, as we observe in §X, can in principle be composed; §VII-F shows that even on this single plan their relative ordering depends on ϵ.

D. Differential Privacy Tradeoffs

We evaluate the impact of the privacy parameter ϵ on execution cost.

Lower values of ϵ provide stronger privacy guarantees but introduce more noise, increasing padding overhead. Figure 4 plots the same sweep on a log-log axis with error bars.

TABLE IV
Effect of ϵ on simulated cost

ϵ	Avg Cost
0.01	100,107,299
0.10	16,162,952
0.50	8,918,789
1.00	7,976,465
5.00	7,251,214

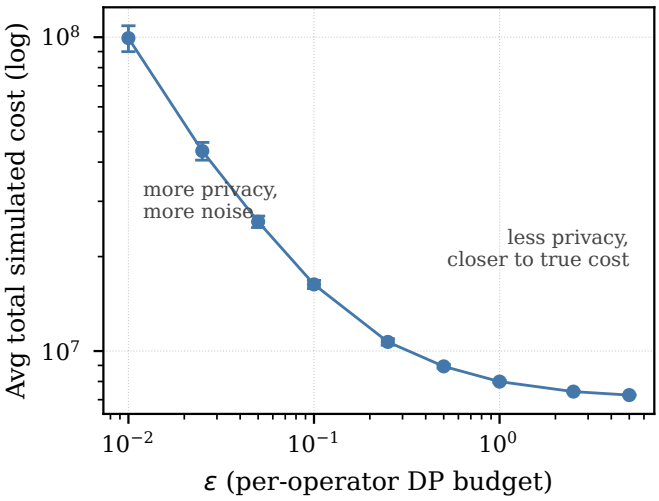


Fig. 4. Average total simulated cost under DPResize as ϵ varies, averaged over 20 trials per point. Smaller ϵ buys stronger privacy at the cost of more Laplace noise and consequently more padding.

E. Scaling Behavior

We also evaluate how the policies scale with increasing input size.

TABLE V
Scaling behavior by policy

Input Rows	FullOblivious	DPResize	ORQ
1,000	6.3×10^{10}	1.7×10^{5}	7.1×10^{6}
10,000	8.3×10^{13}	8.0×10^{6}	7.0×10^{8}
50,000	1.2×10^{16}	1.8×10^{8}	1.8×10^{10}

The fully oblivious policy exhibits cubic-style growth due to Cartesian padding, while DPResize remains comparatively stable. Figure 5 plots the same three policies on a log-log axis as the input scale grows from 1,000 to 50,000 rows.

F. DPResize / ORQ crossover under varying ϵ

Choosing the privacy budget ϵ is itself a hard problem: Dwork et al. [12] report no community consensus on how practitioners should set it, and argue that a system's ϵ and its consequences should be made explicit rather than buried in implementation details. PrivQ contributes to that goal in the small, by making the cost consequence of a chosen ϵ directly observable on a concrete plan. The

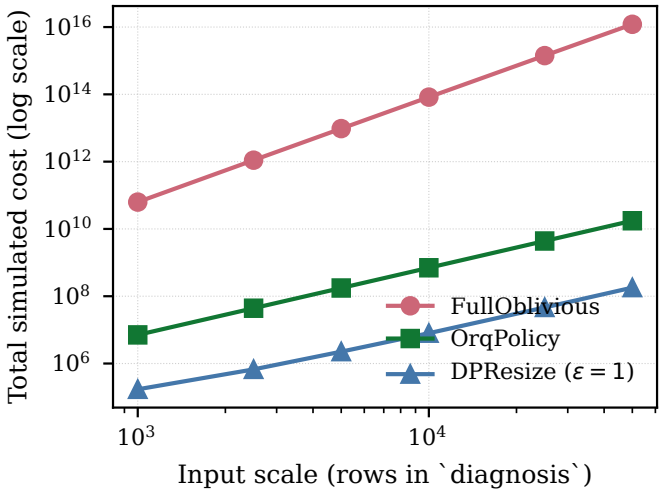


Fig. 5. Total simulated cost as a function of input scale, for the three padding policies. The fully-oblivious cost grows roughly cubically, the ORQ-bound cost grows roughly quadratically, and the DP-resize cost grows roughly linearly with the input.

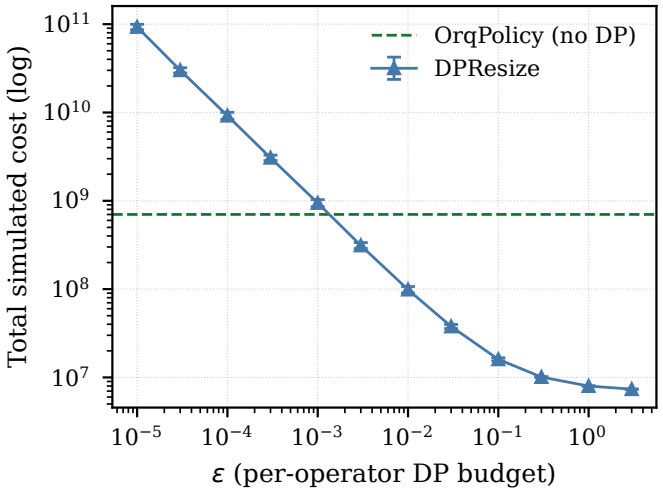


Fig. 6. Total simulated cost on the Listing 9 plan, sweeping the per-operator DP budget ϵ for DPResize and showing OrqPolicy as a horizontal reference. The two policies cross near $\epsilon \approx 1.5 \times 10^{-3}$: ORQ wins below the crossover, DPResize wins above it. Error bars are one standard deviation over 50 trials.

Table III headline compares DPResize at $\epsilon = 1$ against OrqPolicy and shows DPResize cheaper by roughly two orders of magnitude. That ordering is not invariant: as ϵ shrinks, the Laplace scale $1/\epsilon$ grows and the noisy resize $|O|_{\text{true}} + \text{Lap}(\Delta/\epsilon)$ eventually exceeds the deterministic ORQ bound $|A| + |B|$, at which point the policy ordering flips. The crossover budget ϵ^* is not directly given by the closed-form cost equations; it depends on the plan's true join cardinalities, the sensitivity at each join, and the cost-model coefficients. Locating it numerically is exactly the kind of question the simulator is well-suited to answer.

Figure 6 sweeps the per-operator budget ϵ over five orders of magnitude on the Listing 9 plan, averaging 50 trials per point. The OrqPolicy line is a constant horizontal at 7.0×10^8 (no randomness). The two lines cross near $\epsilon \approx 1.5 \times 10^{-3}$: at $\epsilon = 10^{-3}$ the DP-resize cost is roughly 1.3× the ORQ cost and ORQ is cheaper, while at $\epsilon = 5 \times 10^{-3}$ the DP-resize cost is 0.27× ORQ and DPResize is cheaper. The crossover is therefore both budget-dependent and confined to a regime an order of magnitude below what one would consider a normal Shrinkwrap operating point. This is the kind of regime change that motivates running a simulator rather than reading the cost model alone: the cost model implies the crossover exists; the simulator tells you where.

VIII. End-to-End Example

To make the composition of PrivQ's two layers concrete, we walk through a single end-to-end scenario: a researcher wants to know how many heart-disease patients in their cohort have been prescribed aspirin, but their data is held by a federated MPC backend and they want to bound the cost of the secure execution before submitting the query.

a) Step 1: write the plan: The researcher expresses the query as a QueryPlan, populating selectivity and multiplicity hints from offline catalog statistics. The plan is shown in Listing 9.

b) Step 2: compare execution strategies: The researcher runs the plan under all three policies to see what the secure-execution cost would be:

```
for pol in [FullObliviousPolicy(),
            DPResizePolicy(eps=1.0, delta=1e-6),
            OrqPolicy()]:
    print(plan.simulate(tables=tables,
                        policy=pol))
```

Listing 10. Comparing policies on the plan in Listing 9.

This produces three SimulationReport objects, each listing per-operator true cardinalities, padded cardinalities under the policy, and per-operator cost. The headline finding (Table III) is that fully oblivious execution is roughly 10^7 times more expensive than DP-resize on this plan, and that ORQ's join-aggregation bound sits between the two.

c) Step 3: emit DP-bounded SQL: Having decided to deploy with DP-resize, the researcher uses the noisy cardinalities from the simulator to issue a SQL query with a calibrated LIMIT:

```
sql = ("SELECT pid FROM diagnosis "
       "WHERE diag = 'hd'")
true_count = report.steps[0].true_card
p.dp_resize(sql, true_count,
            epsilon=1.0, delta=1e-6)
# -> "SELECT pid FROM diagnosis
#     WHERE diag = 'hd' LIMIT 112"
```

Listing 11. Emitting a DP-resized SQL query.

The returned SQL is safe to issue to the federated backend: the LIMIT bound is a (ϵ, δ)-DP release of the filter's output cardinality (per Eq. 2), and exceeds the true count with probability $1 - \delta$.

IX. Limitations

PrivQ is intended as a teaching and experimentation framework rather than a production MPC engine.

Several important limitations should be noted.

- PrivQ does not execute real MPC protocols; the cardinalities and costs it reports are model estimates, not measurements.
- SQL rewriting transformations operate on query text only. The shrinkwrap() structural pads obscure query shape from a text-channel observer but do not bound intermediate-result leakage from a secure execution engine.
- Simulated execution costs use the unit cost model of Bater et al. [1] (Table 2). Numbers reported by the simulator are meaningful as ratios across policies on the same plan, not as predictions of MPC wall-clock time.
- DPResizePolicy samples noise from a correct implementation of the truncated Laplace mechanism (Eq. 2) but does not enforce a global (ϵ, δ) budget across a plan's operators. The privacy cost of a full plan with ℓ operators is therefore $(\ell\epsilon, \ell\delta)$ under sequential composition; an optimal per-operator budget split (Bater §5) is left for future work.
- The simulator currently supports only linear chains of Filter, Join, and Aggregate operators. Tree-structured plans, additional operator types, and SQL→plan lowering are planned extensions.

These limitations are documented honestly so that users and reviewers can calibrate PrivQ's claims. The framework provides a useful tool for reasoning about the performance and privacy tradeoffs of secure relational query processing, but it should not be confused with a deployment-ready MPC engine.

X. Related Work

Secure relational query processing has received significant attention in both database systems and cryptography research. PrivQ draws directly on two recent systems papers.

Shrinkwrap [1] introduced differential-privacy-based resizing for secure relational query execution in private data federations. By releasing noisy intermediate cardinalities through a truncated Laplace mechanism (Eq. 2), Shrinkwrap reduces the overhead associated with worst-case padding while retaining formal (ϵ, δ)-differential-privacy guarantees on the released sizes. The paper additionally introduces a convex optimization formulation for splitting a global privacy budget across the operators of a query plan (Bater §5), which PrivQ models only at the per-operator level.

ORQ [2] introduced an oblivious join-aggregation operator together with scalable oblivious sorting protocols for secure relational analytics in the outsourced setting. ORQ avoids the cascading Cartesian blowup of naive oblivious joins by sorting the concatenated inputs and computing aggregations in the same oblivious pass (Baum §3.3), yielding $O(n \log n)$ end-to-end complexity for a wide class of analytical workloads. ORQ also reports the first end-to-end MPC results on the complete TPC-H benchmark.

Beyond the secure-query setting, differential privacy has been applied to related data-sharing problems in directions PrivQ does not currently model. Atlas-X [3] obfuscates a bank's daily-published inventory "axe" list so that the trading activity of a single concentrated client is statistically hidden, and reports the first DP deployment live in production in the financial sector. Two features place it on a different axis from Shrinkwrap, ORQ, and PrivQ: it operates under continual observation, releasing a fresh aggregate every day so that the privacy budget accumulates over a time series rather than over the operators of one static query plan, and it sanitizes the output of a single trusted publisher rather than the intermediate cardinalities of a secure-computation transcript. PrivQ's per-operator budget accounting (§IV) is the static-plan analogue of the continual-observation accounting Atlas-X performs over time; extending the simulator's budget model from a single plan to a stream of repeated queries, with the continual-observation accountant of Atlas-X as a reference point, is a natural direction for future work (§XII).

a) Composability of Shrinkwrap and ORQ: A point worth emphasizing is that Shrinkwrap and ORQ attack different terms of the secure-execution cost equation. Shrinkwrap reduces padding by trading a calibrated (ϵ, δ) amount of cardinality leakage for shorter secure arrays; ORQ reduces padding by changing the execution strategy so that a smaller secure array is needed in the first place. The two approaches are orthogonal and, in principle, composable: an ORQ-style execution engine could employ Shrinkwrap-style DP-resize between stages where join-aggregation cannot be applied, achieving the benefits of both. PrivQ's simulator can compare these policies directly, and we view exploring composed policies as a natural extension (§XII).

PrivQ differs from both systems by focusing on educational modeling and lightweight experimentation rather than on cryptographic execution. The padding policies in PrivQ's plan-layer simulator are direct, deliberately simplified reductions of the strategies described in Shrinkwrap and ORQ.

XI. Conclusion

This work presented PrivQ, a Python framework for modeling privacy-preserving relational query execution strategies inspired by Shrinkwrap and ORQ. PrivQ combines a SQL rewriting layer with a plan-layer simulator

for studying oblivious execution, differential-privacy-based resizing, and join-aware aggregation strategies. Experimental results demonstrate the dramatic performance implications of fully oblivious query execution and highlight the effectiveness of Shrinkwrap-style and ORQ-style optimizations.

XII. Future Work

Future work falls along three directions that build directly on the current artifact:

- Global DP budget accounting. The most immediate gap (§IV) is that DPResizePolicy consumes its budget independently at every operator. A global accountant, starting with naive sequential composition and extending to the convex per-operator budget allocation of Bater §5, would let PrivQ report a single (ϵ, δ) for a whole plan and allow honest comparison against ORQ at a chosen budget.
- Cost-based query optimization. The simulator already exposes the cost model in closed form (§V-E); a planner that enumerates equivalent plan rewrites (join reorderings, filter pushdowns) and selects the cheapest under a chosen policy is a small extension and would let PrivQ recommend plans rather than just score them.
- Real MPC-backend integration. Wiring PrivQ's plan layer to an MPC runtime (e.g., MP-SPDZ, EMP-toolkit, or the ORQ codebase itself) would let the simulator validate its predictions against measured execution time and would let plan rewrites chosen in PrivQ be executed without rewriting them in another language.

Longer-term applications (extending the operator set to vector-search and ANN-under-MPC workloads, modeling retrieval-augmented-generation pipelines, and LLM-assisted plan rewriting validated against the simulator) are natural follow-ons but require the budget accountant and the MPC-backend bridge above as prerequisites.

Acknowledgments

The authors would like to thank Dr. Daniel Votipka and the members of the Tufts University Security and Privacy Lab.

References

[1] Johes Bater, Xi He, William Ehrich, Ashwin Machanavajjhala, and Jennie Rogers. Shrinkwrap: Efficient SQL Query Processing in Differentially Private Data Federations. Proceedings of the VLDB Endowment, 12(3):307–320, 2018.

[2] Eli Baum, Sam Buxbaum, Nitin Mathai, Muhammad Faisal, Vasiliki Kalavri, Mayank Varia, and John Liagouris. ORQ: Complex Analytics on Private Data with Strong Security Guarantees. In Proceedings of the 30th ACM Symposium on Operating Systems Principles (SOSP), 2025.

[3] Antigoni Polychroniadou, Gabriele Cipriani, Richard Hua, and Tucker Balch. Atlas-X Equity Financing: Unlocking New Methods to Securely Obfuscate Axe Inventory Data Based on Differential Privacy. In Proceedings of the 23rd International Conference on Autonomous Agents and Multiagent Systems (AAMAS), 2024.

[4] Johes Bater, Gregory Elliott, Craig Eggen, Satyender Goel, Abel Kho, and Jennie Rogers. SMCQL: Secure Querying for Federated Databases. Proceedings of the VLDB Endowment, 10(6):673–684, 2017.

[5] Nikolaj Volgushev, Malte Schwarzkopf, Ben Getchell, Mayank Varia, Andrei Lapets, and Azer Bestavros. Conclave: Secure Multi-Party Computation on Big Data. In Proceedings of the 14th EuroSys Conference, 2019.

[6] Anders Dalskov, Daniel Escudero, and Ariel Nof. Fast Fully Secure Multi-Party Computation over Any Ring with Two-Thirds Honest Majority. In Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS), 2022.

[7] Antigoni Polychroniadou, Gilad Asharov, Benjamin Diamond, Tucker Balch, Hans Buehler, Richard Hua, Suwen Gu, Greg Gimler, and Manuela Veloso. Prime Match: A Privacy-Preserving Inventory Matching System. In Proceedings of the 32nd USENIX Security Symposium, 2023.

[8] David Byrd and Antigoni Polychroniadou. Differentially Private Secure Multi-Party Computation for Federated Learning in Financial Applications. In Proceedings of the First ACM International Conference on AI in Finance (ICAIF), 2020.

[9] Vaikkunth Mugunthan, Antigoni Polychroniadou, David Byrd, and Tucker Balch. SMPAI: Secure Multi-Party Computation for Federated Learning. In NeurIPS 2019 Workshop on Robust AI in Financial Services, 2019.

[10] Sankha Das, Sayak Ray Chowdhury, Nishanth Chandran, Divya Gupta, Satya Lokam, and Rahul Sharma. Communication Efficient Secure and Private Multi-Party Deep Learning. Proceedings on Privacy Enhancing Technologies, 2024(1):169–189, 2024.

[11] Yue Guo, Antigoni Polychroniadou, Elaine Shi, David Byrd, and Tucker Balch. MicroSecAgg: Streamlined Single-Server Secure Aggregation. Proceedings on Privacy Enhancing Technologies, 2024(3):246–275, 2024.

[12] Cynthia Dwork, Nitin Kohli, and Deirdre Mulligan. Differential Privacy in Practice: Expose Your Epsilons! Journal of Privacy and Confidentiality, 9(2), 2019.

[13] Simson Garfinkel, John M. Abowd, and Christian Martindale. Understanding Database Reconstruction Attacks on Public Data. Communications of the ACM, 62(3):46–53, 2019.

[14] Arvind Narayanan and Vitaly Shmatikov. Robust De-anonymization of Large Sparse Datasets. In Proceedings of the 2008 IEEE Symposium on Security and Privacy (S&P), pages 111–125, 2008.

[15] Krishnamurty Muralidhar and Josep Domingo-Ferrer. Database Reconstruction Is Not So Easy and Is Different from Reidentification. Journal of Official Statistics, 39(3):381–398, 2023.

Figures

For reference, we summarize each figure and the role it plays in the paper.

- Figure 1 sketches the secure multi-party computation setting that Shrinkwrap, ORQ, and the systems of §III share. Data owners secret-share private inputs to non-colluding compute parties, which evaluate the query obliviously over shares before the result is reconstructed and returned. The figure orients the reader to the layer PrivQ models: the cardinality and cost behavior of the oblivious-evaluation step, not the cryptography beneath it.
- Figure 2 shows PrivQ's two-layer architecture. The SQL rewriting layer applies four text-level transformations to a query string, and the plan-layer simulator evaluates a relational QueryPlan under three padding policies to produce a SimulationReport. The figure makes explicit that the two layers consume

different inputs (SQL text versus a programmatic plan) and share only the C++ primitives described in §V.

- Figure 3 plots true versus padded intermediate cardinalities at each operator of the three-way healthcare plan (Listing 9) under fully oblivious execution. On a logarithmic axis, the gap between the true and worst-case-padded sizes spans more than ten orders of magnitude by the second join, visualizing the $n \to n^2 \to n^3$ cascading blowup that motivates the whole paper.
- Figure 4 shows average total simulated cost under DPResize as the privacy budget ϵ varies, averaged over 20 trials per point on a log-log axis with error bars. It quantifies the privacy-utility tradeoff: smaller ϵ buys stronger privacy at the cost of larger Laplace noise and therefore more padding.
- Figure 5 compares total simulated cost for the three policies as the input scale grows from 1,000 to 50,000 rows. The fully oblivious cost grows roughly cubically, the ORQ-bound cost roughly quadratically, and the DP-resize cost roughly linearly, separating the policies by asymptotic behavior rather than by a single operating point.
- Figure 6 sweeps the per-operator DP budget ϵ for DPResize against OrqPolicy shown as a horizontal reference, averaging 50 trials per point. The two policies cross near $\epsilon \approx 1.5 \times 10^{-3}$: ORQ is cheaper below the crossover and DPResize above it. This is the regime change that the closed-form cost model implies but does not locate, and which the simulator is built to find.